COMPONENTS

Admonitions

Four admonition flavours — `note`, `tip`, `warning`, `danger`. The first three sit on a warm surface tone with a left rule whose weight encodes the intent. The fourth inverts to ink so it cannot be skimmed past.

NOTE

A quiet, non-load-bearing aside. Border-left in `--ink3`, surface background, label in `ink`.

> **NOTE**
>
> PEP 703 introduces a no-GIL build of CPython. Until it ships as the default, reason about thread-pool sizing as if the GIL is there. Inline code like `ThreadPoolExecutor` keeps the body sans–mono contrast.

TIP

Same surface tone as `::: note`, but the left rule darkens to full ink — "do this if you want the better outcome", not "stop and read".

> **TIP**
>
> For I/O-bound work, start with `min(32, os.cpu_count() + 4)` workers and measure before tuning. The default in `concurrent.futures` is rarely the bottleneck; the work *queue depth* almost always is.

WARNING

A distinct warmer surface (`--surface2`), `ink2` left rule, **plus** hairlines top and bottom. The extra borders elevate the block visually without spending color.

> **WARNING**
>
> A pool that hands out threads faster than the downstream API can absorb them is just a denial-of-service generator pointed at your own infrastructure. Add a bounded queue and a backpressure signal before you raise `max_workers`.

DANGER

The single inversion event in the entire system. Ink fill, page-coloured text, no border. It earns its weight by being the only place colour flips.

DANGER

Never call `pool.shutdown(wait=False)` from inside a task that the same pool is executing — the worker will block forever waiting for itself to finish. This is a deadlock that no timeout can break, only a process kill.

STACKED

Admonitions read cleanly when stacked; the rule weights and surface tones do the differentiation, so no extra spacing is needed between them.

> NOTE
>
> First, the quiet aside.

> WARNING
>
> Then the elevated caution.

DANGER

Then the inversion.